Topologica-Sort

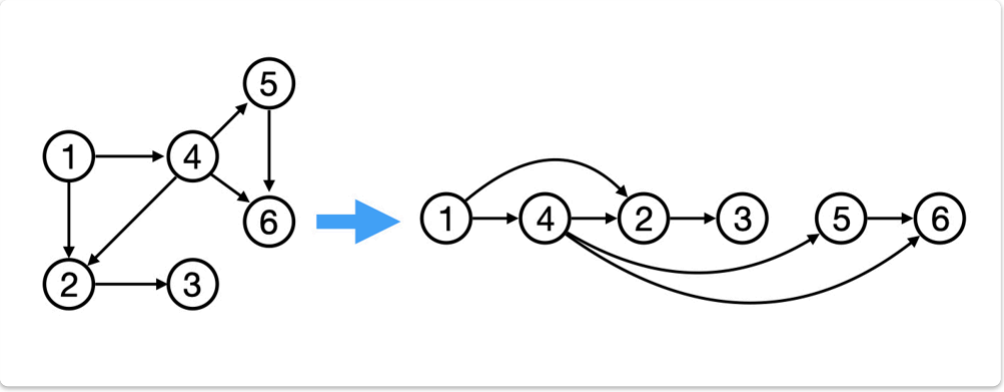
Topological Sort (Ordenación Topológica)

La ordenación topológica se aplica sobre grafos dirigidos acíclicos (DAG: *Directed Acyclic Graph*).

Su objetivo es producir un orden lineal de los vértices tal que, para cada arista dirigida ((u → v)), el nodo `u` aparezca antes que `v` en el orden resultante.

Interpretación típica:

- Cada nodo = una tarea.
- Cada arista `(A → B)` significa: A debe hacerse antes que B.



Ejemplo (proyecto de construcción):

> *cimientos → pilares → paredes → electricidad → pintura*

Una ordenación topológica te da una secuencia válida que respeta todas las dependencias.

Importante:

- Solo existe ordenación topológica si el grafo no tiene ciclos.
- Si hay un ciclo (A depende de B, B de C, y C de A), entonces no existe orden lineal que cumpla todas las restricciones.

Complejidad:

- Tiempo: (O(V + E))
  (se visita cada vértice y cada arista un número constante de veces).
- Espacio: (O(V + E))
  (para la lista de adyacencia, grado de entrada, cola, etc.).

Esquema paso a paso

Vamos a usar el algoritmo clásico de Kahn (basado en grado de entrada + cola).

Idea intuitiva con tareas:

1. Representas las tareas como nodos de un grafo dirigido.
2. Cada dependencia `(A antes que B)` se convierte en una arista `A → B`.
3. Las tareas que no dependen de ninguna otra (nadie apunta hacia ellas) son las que pueden hacerse primero.
   Es decir, son los nodos con grado de entrada 0.
4. El algoritmo va:
   - Eligiendo uno de esos nodos sin dependencias pendientes,
   - Añadiéndolo al orden,
   - "Eliminándolo" del grafo junto con sus aristas de salida.
5. Al "eliminar" sus aristas, puede ocurrir que otras tareas se queden con grado de entrada 0, y pasen a ser las siguientes en la lista de candidatos.

Pasos detallados del algoritmo (Kahn)

Supongamos un grafo dirigido `G = (V, E)` representado con listas de adyacencia:

- `V`: conjunto de vértices (tareas).
- `E`: conjunto de aristas dirigidas `(u → v)` (dependencias).

1. Calcular grados de entrada

Para cada vértice `v` en `V`, calculamos su grado de entrada, es decir, cuántas aristas llegan a él.

- Inicializar `inDegree[v] = 0` para todos los vértices.
- Para cada arista `(u → v)`:
  - Hacer `inDegree[v]++`.

Interpretación:

- `inDegree[v]` = cuántas tareas deben hacerse antes de `v`.

2. Inicializar la cola con vértices sin dependencias
   - Crear una cola (queue) `Q`.
   - Insertar en `Q` todos los vértices `v` tales que `inDegree[v] == 0`. Son las tareas que pueden empezar ya, porque no dependen de ninguna otra.
3. Proceso principal
   - Crear una lista vacía `order` que contendrá el orden topológico.
   - Mientras la cola `Q` no esté vacía:
     1. Sacar un vértice `u` de `Q` (por ejemplo, el primero).
     2. Añadir `u` al final de `order`.
     3. Para cada vecino `v` de `u` (cada arista `u → v`):
        - Reducir su grado de entrada: `inDegree[v]--`.
        - Si tras restar, `inDegree[v] == 0`, entonces:
          - Encolar `v` en `Q` (ya no tiene dependencias pendientes).
4. Comprobar si había ciclos
   - Cuando la cola se vacía, revisamos si hemos añadido todos los vértices en `order`.
   - Si `len(order) == número_de_vértices`, entonces:
     - ¡Perfecto! `order` es una ordenación topológica válida.
   - Si no:
     - Significa que quedan vértices con grado de entrada > 0 → existe al menos un ciclo.
     - En ese caso, no existe orden topológico posible.

Relación con tu idea de “tachar” elementos

Otra forma de pensarlo (lo que anotabas en tus apuntes):

1. Haces una lista de parejas `(origen, destino)` que representan las aristas.
2. Buscas un vértice que **nunca aparezca como destino** (nadie depende de él por delante) → ese puede ser el primero.
3. Lo añades al orden y “tachas” todas las parejas en las que aparezca como origen.
4. Repites el proceso con el grafo reducido.

Eso es conceptualmente lo mismo que:

- Calcular grados de entrada.
- Ir cogiendo vértices con grado 0.
- Eliminar sus aristas y actualizar los grados.

Pseudocódigo (estilo CLRS / Kahn)

```
TOPOLOGICAL-SORT(G)
    V = conjunto de vértices de G
    Adj = listas de adyacencia de G   // Adj[u] = vecinos v con arista (u → v)

    para cada vértice v en V
        inDegree[v] = 0

    // Calcular grado de entrada de cada vértice
    para cada vértice u en V
        para cada v en Adj[u]
            inDegree[v] = inDegree[v] + 1

    Q = cola vacía

    // Encolar todos los vértices con inDegree 0
    para cada vértice v en V
        if inDegree[v] == 0
```

```
            ENQUEUE(Q, v)

    order = lista vacía

    mientras Q no esté vacía
        u = DEQUEUE(Q)
        añadir u al final de order

        para cada v en Adj[u]
            inDegree[v] = inDegree[v] - 1
            if inDegree[v] == 0
                ENQUEUE(Q, v)

    if longitud(order) == número_de_vértices
        return order        // orden topológico válido
    else
        // Quedaron vértices con inDegree > 0 → ciclo
        reportar "Ciclo detectado: no hay orden topológico."
```

Implementación en Python

Vamos a suponer que el grafo viene representado como:

- `graph` : diccionario `nodo → lista de vecinos` .
- `graph.keys()` contiene todos los vértices.

```python
from collections import deque

def topological_sort(graph):
    """
    Ordenación topológica usando el algoritmo de Kahn.
    graph: diccionario {nodo: [lista_de_vecinos]} que representa un DAG.
    Devuelve una lista con un orden topológico de los nodos.
    Si hay un ciclo, lanza ValueError.
    """
    # 1. Calcular el grado de entrada de cada nodo
    in_degree = {node: 0 for node in graph}
```

```python

    for u in graph:
        for v in graph[u]:
            in_degree[v] = in_degree.get(v, 0) + 1

    # Asegurar que todos los nodos están en in_degree (incluidos aislados)
    for node in graph:
        in_degree.setdefault(node, 0)

    # 2. Inicializar cola con nodos de grado de entrada 0
    queue = deque([node for node in in_degree if in_degree[node] == 0])

    order = []

    # 3. Proceso principal
    while queue:
        u = queue.popleft()
        order.append(u)

        for v in graph.get(u, []):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)

    # 4. Comprobar si hubo ciclo
    if len(order) != len(in_degree):
        raise ValueError("El grafo contiene un ciclo. No existe ordenación topológica.")

    return order


if __name__ == "__main__":
    # Ejemplo de DAG
    graph = {
        "cimientos": ["p1"],
        "p1": ["p2"],
        "p2": ["p3"],
        "p3": ["electricidad", "placas"],
```

```
        "electricidad": [],
        "placas": []
    }

    orden = topological_sort(graph)
    print("Orden topológico:", orden)
```